

Watopoly: Design Document

Ian Mairena Jarquin, Luke Bennet, Justin Barrette

1. Introduction

This document serves as the final software design specification for "Watopoly," a terminal-based clone of the classic board game Monopoly, implemented in C++ using object-oriented programming (OOP) principles. The project, developed as the final assignment for CS246, focuses on delivering a robust, maintainable, and extensible system architecture. The core design heavily relies on fundamental OOP concepts such as encapsulation, inheritance, polymorphism, and established design patterns to ensure high cohesion and low coupling across the system components. The primary goal of this document is to detail the architectural choices made, demonstrate how they adhere to best practices in software design, and prove the resilience of the system to future modifications.

2. Overview

The Watopoly architecture is designed around three central components: the **Game** class, the **Board** class, and the **Square** class hierarchy, arranged to manage the flow and state of the game and the players.

The **Game class** acts as the system's controller, managing the main game loop, sequencing player turns, processing user commands, and handling state transitions such as auctions and bankruptcies. This high-level responsibility allows for high cohesion and is isolated from the specifics of individual game elements.

The **Board class** serves as the central manager for all physical squares in the game. It is responsible for initializing and maintaining the ordered collection of **Square** objects throughout the game. It also provides a consistent interface for the **Game** and **Player** classes to interact with the game map.

The **Square class** hierarchy consists of properties that can't be owned by players (CollectOSAP, GoToTims, GooseNesting, etc), properties that can be owned by players (academic buildings, gyms, and residences), and properties involving drawing cards (SLC and Needles Hall). It uses **polymorphism** to handle the effects of landing in each individual square on the board.

This approach allows the **Game** class to interact with any type of board space (e.g. Academic Building, Non-Property, SLC/Needles Hall) through a unified interface, decoupling the game logic from the specific actions of each square.

Moreover, the **Player** class encapsulates player-specific state, including cash amount, owned properties, current board position, money owed, turns left in Tims, etc. The player object is designed to be self-contained, managing its assets and liabilities, and interacting with the Game class only for high-level actions (e.g. Getting a player's name or cash amount).

Deviation from Due Date 1 UML Plan: The final design largely remained consistent with the initial UML plan. The most significant architectural decisions including the central role of the **Game** and **Board** classes, the **Square** inheritance hierarchy, and the role of the **Player** class, were maintained to preserve system stability and modularity.

3. Design:

Inheritance and Polymorphism

We utilized a deep **inheritance** hierarchy rooted in the abstract base class **Square**. This structure is essential for modelling the diverse set of squares on the Watopoly board:

- **Square** (Abstract Base Class): Defines the minimal common interface for all spaces.
 - **Property**: Base class for purchasable squares.
 - **AcademicBuilding**: Derived from **Property**, includes logic for improvements, mortgage, and calculating tuition based on number of improvements.
 - **Residence**: Derived from **Property**, implements rent calculation logic based on the number of residences owned by the creditor.
 - **Gym**: Derived from **Property**, implements fee calculation logic based on the dice roll that caused the player to land on the Gym square, and number of gyms owned by the creditor.
 - **CardSquare**: Base class for squares that cannot be bought (e.g., Go to Tims Line, SLC).
 - **SLC** and **NeedlesHall**: Derived from **CardSquare**, keeps track of **Cards** and when to apply them.
 - Other squares (**CoopFee**, **GoToTims**, **GooseNesting**): Derived from **Square**, each produces a unique effect when landed on.

The basis of this design is the pure virtual method:
virtual void landOn(Player* p) = 0;

This method is implemented by every concrete `Square` class. In the game loop, the `Game` class simply calls `currentSquare->landOn(currentPlayer)`, without needing to know whether the player is landing on an Academic Building, an SLC square, or a Residence. This application of polymorphism vastly reduces **coupling** between the `Game` controller and the individual `Square` implementations.

Cohesion and Coupling Analysis

Maximizing Cohesion: As mentioned in the “Overview” section of this document, we focused on dividing the logic of the game into three highly focused modules. This decision helped us achieve high cohesion throughout the Watopoly project:

- The `Game` class acts exclusively as the controller of the overall game, handling command parsing, player turns, and complex multi-player events (like auctions and bankruptcy).
- The `Board` is strictly responsible for reading the configuration files (board.txt, squares.txt, and cards.txt) and populating the grid.
- The `Square` hierarchy focuses purely on the localized, immediate effects of a player landing on a specific square.

By isolating these responsibilities, debugging and expanding individual features (such as adding new console commands, adding more cards that do other things, or changing the functionality of a specific square) did not conflict with the implementation of the other classes.

Intentional Coupling (Player, Property, and Square): While we successfully minimized coupling between the high-level controllers, we tightly coupled the `Player`, `Square`, and `Property` classes into one module. In our design, a `Player` holds a `vector<Property*>` to track its properties, while a `Property` simultaneously holds a `Player* owner` pointer to track who collects rent. Furthermore, the polymorphic `Square::landOn(Player* p)` method requires direct knowledge of the `Player` interface to modify their cash balance or position on the board.

The Benefit of Tight Coupling for Watopoly: Although high coupling is usually seen as a “red flag” in Object-Oriented programs, we deliberately chose this design because it accurately models the connections among the mechanics of Watopoly while improving code performance and readability.

Because a `Player` maintains direct pointers to its `Property` objects, implementing player-centric commands becomes incredibly efficient. For example, the `assets()` method requires no external dependencies; the `Player` class simply iterates through its internal vector of properties and uses `prop->getName()` and `prop->isMortgaged()` to display the player's assets. If we had artificially forced low coupling—such as requiring players to store only the integer indices of their properties—every asset query, rent calculation, or monopoly verification would require calling the `Board` or `Game` class to act as a middleman to fetch the data, creating high coupling between the high-level classes. Thus, allowing `Player` and `Property` to communicate directly kept the code logic localized to the objects actually involved in the events.

4. Resilience to Change

The system's architecture, particularly its reliance on polymorphism, is highly resilient to changes in game specification.

Extending Square Hierarchy

The polymorphic `Square` hierarchy ensures that adding new types of board spaces requires minimal modification to the existing, core game logic.

Scenario: Adding a new "ION Station" building type

1. **Extension:** A new class, `IONStation`, would be created, publicly derived from `Property`.
2. **Implementation:** `IONStation` would implement the specific logic for fee calculation and the `landOn(Player* p)` method.
3. **Minimal Modification:** The `Game` class's core loop remains entirely unchanged. It will continue to call `square->landOn(player)`, which will redirect the call to the new `IONStation::landOn` implementation.
4. **Configuration Update:** Only the `Board` builder/initializer must be updated to create an `IONStation` object at the correct index, rather than an existing square type.

This approach ensures that the central, tested `Game` loop stays untouched, while the system's functionality is extended through new classes.

New Commands

User interaction is handled via a command harness that maps strings (e.g., "roll", "trade", "mortgage") to actions within the `Game` class.

Scenario: Adding a new "cheat" command

To add a new player command, one only needs to:

1. Add the new command string to the command harness `Game::playTurn(bool)`.
2. Implement the corresponding private method (e.g., `Game::handleCheat()`) within the `Game` class.

No changes are required to the main input loop or existing command handlers, demonstrating the architectural flexibility in accommodating new commands.

Changing Game Rules (Dice / Tims Logic)

The centralization of turn logic within the `Game` class ensures that rule modifications can be implemented cleanly and consistently.

Scenario: Modifying the rules for rolling doubles or exiting the Tims Line

Extension: The rules governing dice rolls or Tims Line behavior are adjusted (e.g., allowing more than three doubles, or changing how players exit Tims).

Implementation: The logic for dice rolling and Tims handling is encapsulated within methods such as `Game::handleRoll` and `Game::handleTimsTurn`. Any rule changes would be implemented directly within these methods.

Minimal Modification: No changes are required to the `Board`, `Player`, or `Square` classes. The `Game` class continues to control the flow of turns and enforce rules centrally.

This approach ensures that rule changes remain localized, reducing the risk of unintended side effects across the system.

Board, Squares, and Cards

In our implementation, the `Board` class is in charge of creating and managing the board display and the squares on it (including card squares). However, to maximize adaptability to change, we fetch the board, squares, and cards data at runtime during the initialization of the board.

Furthermore, we only make minimal assumptions about this data when reading from it (for example that the displayed square size will remain constant, or that there will be at least one card managed by each card square (SLC and NeedlesHall)).

Notably, we do not make any assumptions about the positions of the squares and how many of each will exist (except for the DCTimsLine), which academic buildings will be present, their names, prices, and to which monopolies they belong, the effects of cards found in the card squares, etc.

This allows for a very large degree of freedom with how the game is set up, and the fact that this all happens at runtime means it can be implemented as a command line argument (see the Extra Features section).

5. Answers to Questions

Question A: After reading this [Board] subsection, would the Observer Pattern be a good pattern to use when implementing a gameboard? Why or why not?

No, not in our case.

The Observer Pattern could be a suitable design choice for implementing a gameboard, particularly in systems where multiple components need to react to changes in the game state. In this pattern, the `Board` would act as the subject, while display components (such as a text-based renderer or graphical user interface) would act as observers. Whenever the state of the board changes, such as player movement, property ownership updates, or improvements, observers would be notified automatically and update their views accordingly. This promotes loose coupling between the game logic and the presentation layer.

However, in our implementation, the Observer Pattern was not necessary. Our system uses a single, terminal-based display where the board is explicitly redrawn after each action using direct method calls (e.g., `board.display()`). Since there is only one view of the board and all updates are controlled centrally by the `Game` class and not by multiple components, introducing the Observer Pattern would add unnecessary complexity without providing significant benefits.

Additionally, our design requires detailed and specific updates to the board state (such as exact player positions and property changes), which are more straightforwardly handled

through direct communication between classes. The Observer Pattern is typically more useful in systems with multiple independent views or with graphical interfaces (such as in Assignment 4).

Therefore, while the Observer Pattern is conceptually applicable, it was not an appropriate choice for our specific design.

Question B: Suppose that we wanted to model SLC and Needles Hall more closely to Chance and Community Chest cards. Is there a suitable design pattern you could use? How would you use it?

We did not include a specific design pattern in our implementation. However, we designed our solution in a way that encapsulates different behaviors into separate classes and allows them to be used interchangeably through polymorphism, achieving similar flexibility and extensibility of the Chance and Community Chest cards.

In our final implementation, this design is realized through the Card class hierarchy. We define a base class `Card` with a common interface(`apply(Player *p)`), and derive multiple concrete card types such as `MoveCard`, `MoneyCard`, `MoveToCard`, and `GoToTimsCard`. Each subclass encapsulates a specific action that can be applied to a player.

When a player lands on an SLC or Needles Hall square, the `CardSquare::landOn` method selects the current card from a collection and invokes its `apply` method. Due to polymorphism, the correct behaviour is executed depending on the dynamic type of the card, without the calling code needing to know which specific card type is being used.

Additionally, as initially planned, we used a text-based approach to define the cards. The deck of cards is dynamically constructed by reading from an input file(`cards.txt`), where each line contains information such as the card's associated square (`SLC` or `NeedlesHall`), type (e.g., `Move`, `Money`, `MoveTo`, `GoToTims`), quantity, parameter value, and description. This allows the system to be easily extended by modifying card definitions or adding new ones without changing the core game logic.

This design closely mirrors the behaviour of Chance and Community Chest cards in Monopoly. It promotes extensibility and maintains low coupling, since new card behaviors can be introduced by adding new subclasses of `Card` without modifying existing code.

Question C: Is the Decorator Pattern a good pattern to use when implementing Improvements? Why or why not?

No.

The biggest feature of a decorator design pattern is that it allows you to take advantage of the order that the decorators were added. For example in assignment 4, the colour of a pixel was determined by the *first* decorator in the chain that covered the pixel. However, in Watopoly, the calculation that our `payFee()` method must do only depends on the total number of improvements to a `AcademicBuilding`, not the order in which they were added.

The second advantage of using a decorator design pattern is that it allows you to have an unlimited number of decorations per object. However, this is again unnecessary, as each `AcademicBuilding` can only have at most five improvements.

With this in mind, we decided not to implement the decorator design pattern for improvements, and instead, we will simply store a field `int numImprovements` inside of our `AcademicBuilding` class.

6. Extra Credit Features

Memory Management with Smart Pointers and STL containers

This Watopoly project was implemented to be completely memory-leak free through the disciplined use of C++ **smart pointers**, specifically `std::unique_ptr` and `std::vector<>`.

Technical Approach and Challenges:

1. **Ownership Semantics:** We used `std::unique_ptr` for objects with exclusive ownership, such as the `Square` objects managed by the `Board`. The `Board` class owns all `Squares`, and when the `Board` is destroyed, all squares are automatically cleaned up, eliminating manual `delete` calls and ensuring RAII (Resource Acquisition Is Initialization).
2. **Polymorphism and Containers:** A primary challenge was storing polymorphic objects (`AcademicBuilding`, `Residence`, etc.) in the `Board`'s container. This was solved by using a container of smart pointers to the base class:

`std::vector<std::unique_ptr<Square>>`. This allows for polymorphic behavior while ensuring correct memory deallocation for the derived objects.

3. **Cyclic References:** The most significant challenge was managing potential cyclic references between a `Player` and the `Property` objects they own. If both held `std::shared_ptr` to each other, neither would be destroyed. We solved this by using regular pointers and having neither object own the other. The `Property` objects are owned by the `Board`, and the players, by the `Game`.

This approach not only prevents memory leaks but also enhances the code's safety and readability by explicitly defining ownership semantics.

Customizable board, squares, and cards

As mentioned in the Resilience to Change section, our program reads its data for the `Board`, `Squares` and `Cards` at runtime during initialization. This allowed us to implement the following three command line arguments.

1. **-board [file]**

This replaces the standard board display (as shown in the Watopoly instructions) with the display contained in the given file. This allows you to customize the characters in the square borders as well as the design in the center of the board. For example, you could insert an ASCII drawing of Waterloo's emblem in the middle of the board.

2. **-squares [file]**

This replaces the default configuration of squares (as shown in the Watopoly instructions) with the configuration in the given file. This allows for a great deal of customization in what squares you have on your board. For example, you could play with a board containing only `SLC` and `NeedlesHall` spaces or one with only `GooseNesting` squares (except for the required `CollectOSAP` and `DCTimsLine` squares).

3. **-cards [file]**

This replaces the default cards and associated probabilities (as shown in the Watopoly instructions) with the cards specified in the given file. This allows for tight control over exactly what types of effects are possible when landing on either an `SLC` or a `NeedlesHall` square. For example, you could have all low-value `MoneyCards` for a calmer game or all high-value `MoveCards` for a more exciting game. Furthermore, this can be used in combination with the **-squares** command line argument to produce some very entertaining results.

SLC and NeedlesHall decks

In our program, we implemented SLC and NeedlesHall cards to function like actual decks of cards. This means the order of cards is randomized, but they will all be played exactly once before the cards are "shuffled". This makes for a more realistic game experience where you cannot, by chance, keep getting the same cards over and over again.

Pairing with this, we added a cards section to our board display, showing how many cards remain in each deck.

7. Final Questions

What lessons did this project teach you about developing software in teams?

Developing Watopoly as part of a team highlighted that the most important phase of software development occurs before a single line of code is written. We found the UML class model to be particularly valuable. It served not only as a blueprint for our code but also as an informal contract between team members to guarantee that when one teammate's code depended on another's, it worked exactly as expected.

This maintained consistent direction for the team and ensured smooth integration of each of our parts. Overall, this focus on a polished UML greatly reduced architectural mistakes and streamlined communication throughout the project.

What would you have done differently if you had the chance to start over?

If we had the chance to start over, the primary focus would be on finding an efficient way to **eliminate the high coupling** that still exists between the `Square`, `Property`, and `Player` classes regardless of the convenience this brought to the implementation of various methods. An alternate approach could involve the `Player` class maintaining integer counters for the number of gyms and residences owned, enabling the `landOn` methods in the `Gym` and `Residence` classes to function correctly. This would also allow the `Board` class to act as the central manager of all assets without requiring `Player` and `Property` classes to be tightly coupled.

Additionally, we would have integrated a robust test suite early in the development process to check the functionality of core features like saving/loading a game, or declaring bankruptcy. This way, bugs would have been caught earlier and we could have spent more time implementing new features rather than debugging current ones.

8. Conclusion

This Watopoly project successfully demonstrated the application of object-oriented principles. The resulting C++ program exhibits a stable and maintainable architecture, largely due to the disciplined use of inheritance, polymorphism, and smart pointers. The design's inheritance structure, particularly within the `Square` hierarchy, ensures that the system is highly resilient to future specification changes. The separation of concerns, with the `Game` class acting as the central controller and specific game elements encapsulated in their respective classes, has resulted in a final product with high cohesion and low coupling. Lastly, in completing this project as a team, we have refined our communication skills as well as our coding skills and earned a deeper understanding of what it's like to be a developer.